

PAYLINE FINANCIAL DASHBOARD

Cuestionario Técnico Exhaustivo: Análisis Integral de Backend, Frontend, Base de Datos, Seguridad y Flujo P2P

Este documento contiene un banco de **58 preguntas y respuestas técnicas de nivel profesional** que analizan en profundidad la creación, estructura del código fuente, decisiones arquitectónicas, esquema relacional en PostgreSQL/Supabase, políticas de seguridad RLS, procedimientos almacenados PL/pgSQL, reactividad con Angular Signals y resolución de errores reales afrontados durante el desarrollo del proyecto.

Módulo	Temática Central	Nº Preguntas
Módulo 1	Visión General y Arquitectura del Proyecto	7 (1 - 7)
Módulo 2	Base de Datos y Modelo Relacional en PostgreSQL	11 (8 - 18)
Módulo 3	Seguridad, Autenticación y Políticas RLS	10 (19 - 28)
Módulo 4	Lógica de Negocio y Funciones PL/pgSQL	7 (29 - 35)
Módulo 5	Frontend en Angular 19: Componentes Standalone	8 (36 - 43)
Módulo 6	Gestión del Estado con Signals y Reactividad Moderna	6 (44 - 49)
Módulo 7	UI/UX, Gestos Táctiles, Tokens SCSS y Accesibilidad	5 (50 - 54)
Módulo 8	Depuración, Resolución de Problemas y Buenas Prácticas	4 (55 - 58)

Proyecto: Payline Financial App & Dashboard | **Entorno:** Angular 19 (Standalone + Signals) + Supabase (PostgreSQL 15)

Estado del Código: 100% Funcional, Compilación Limpia, RLS y P2P Operativo | **Fecha:** Septiembre 2026

MÓDULO 1: VISIÓN GENERAL Y ARQUITECTURA DEL PROYECTO

Concepción, stack tecnológico, Clean Architecture y estructura modular de carpetas.

PREGUNTA 1

1. ¿Qué es Payline y cuál es el objetivo funcional de la plataforma desarrollada?

Respuesta: Payline es una aplicación web financiera moderna diseñada para la gestión integral de finanzas personales, visualización de métricas de ingresos/egresos, historial transaccional paginado y transferencias de dinero entre usuarios en tiempo real con conversión multdivisa (EUR, USD, GBP, PEN). Su objetivo es proporcionar una experiencia de usuario fluida, reactiva e interactiva sin recargas de página, respaldada por un backend BaaS (Backend as a Service) altamente seguro con PostgreSQL en Supabase.

PREGUNTA 2

2. ¿Qué tecnologías principales componen la solución tanto en el frontend como en el backend?

Respuesta: En el frontend se emplea Angular 19 utilizando TypeScript 5+, arquitectura zoneless y Signals para reactividad pura, componentes Standalone sin NgModules, control flow nativo (@if, @for, @switch) y SCSS modular con variables nativas. En el backend se utiliza Supabase integrado con PostgreSQL 15+, PL/pgSQL para procedimientos almacenados transaccionales, Row Level Security (RLS) para autorización a nivel de fila y PostgREST para la generación automática de endpoints REST seguros.

PREGUNTA 3

3. ¿Cómo está estructurado el código del frontend siguiendo los principios de Screaming / Clean Architecture?

Respuesta: La aplicación se divide estrictamente en 4 capas dentro de 'src/app':

- **nucleo/ (Core):** Contiene servicios singleton (Supabase, Auth, Transacciones, Usuarios, etc.), modelos de dominio TypeScript, utilidades de formato y errores, y guardas de ruta. No depende de ninguna otra capa.
- **compartido/ (Shared):** Componentes visuales reutilizables y sin estado de negocio (Avatar, Icono, Tarjeta, Esqueleto, Avisos), directivas y tuberías.
- **diseño/ (Layout):** Componentes estructurales de la interfaz como el encabezado, la barra lateral de navegación y el layout principal con router-outlet.
- **funcionalidades/ (Features):** Módulos funcionales organizados por dominio: autenticación (login/registro), transacciones (dashboard, transferencia rápida, historial, volumen de pagos), informes y panel.

PREGUNTA 4

4. ¿Qué ventaja ofrece separar el Núcleo (Core) de las Funcionalidades (Features)?

Respuesta: Permite aislar la lógica de negocio y las llamadas a la infraestructura (Supabase, Auth, PostgREST) en servicios desacoplados de la representación visual. Si en el futuro se reemplaza Supabase por otra API REST o GraphQL, los componentes visuales apenas sufren alteraciones porque solo consumen señales y métodos tipados provistos por la capa de Núcleo.

PREGUNTA 5

5. ¿Cómo se comunican Angular y Supabase sin necesidad de construir un servidor intermedio (como Express o NestJS)?

Respuesta: Se utiliza el SDK oficial '@supabase/supabase-js'. Supabase expone automáticamente la API de PostgREST, que transforma consultas y mutaciones de tablas y llamadas RPC en solicitudes HTTP seguras sobre WebSockets/HTTPS. La seguridad no reside en un servidor intermedio, sino directamente en PostgreSQL mediante Row Level Security (RLS), garantizando que ningún usuario acceda o modifique datos no autorizados.

PREGUNTA 6

6. ¿Qué rol cumple el archivo 'environment.ts' y cómo se gestionan las credenciales?

Respuesta: Define la configuración de conexión del cliente: 'supabaseUrl' (endpoint del proyecto en la nube), 'supabaseAnonKey' (clave pública anónima de PostgREST, segura para clientes web) y 'usuarioDemold' (UUID del usuario demo precargado). Se complementa con 'environment.development.ts' para pruebas locales y desarrollo continuo.

PREGUNTA 7

7. ¿Cómo se orquestó la organización de scripts y migraciones en la carpeta 'backend'?

Respuesta: Las migraciones SQL están estrictamente ordenadas cronológicamente en 'backend/supabase/migraciones/':

- 001_esquema_inicial.sql: Tablas maestras, tipos ENUM, índices y claves foráneas.
- 002_vistas_y_funciones.sql: Vistas complejas y funciones RPC en PL/pgSQL.
- 003_politicas_rls.sql: Grants de esquema y políticas Row Level Security.
- 004_datos_demo.sql: Semillas de comercios, tasas de cambio y usuarios demo.

El script Node.js 'generar-sql-completo.mjs' concatena automáticamente estos archivos en 'instalacion_completa.sql'.

MÓDULO 2: BASE DE DATOS Y MODELO RELACIONAL EN POSTGRESQL

Esquema de tablas, tipos de datos, restricciones defensivas, índices y vistas de base de datos.

PREGUNTA 8

8. ¿Cuáles son las tablas principales creadas en el esquema y qué propósito cumple cada una?

Respuesta: • **usuarios:** Perfiles extendidos vinculados a la autenticación (nombre, correo, rol, moneda base, avatar).

• **comercios:** Catálogo maestro de entidades y plataformas comerciales (Starbucks, Netflix, Apple, etc.).

• **tasas_cambio:** Registros de conversión entre pares de divisas con marca temporal de actualización.

• **contactos:** Destinatarios frecuentes asociados al usuario para transferencias rápidas.

• **transacciones:** Libro mayor de movimientos financieros (ingresos y egresos) con comercio, monto y estado.

• **transferencias:** Registro de transferencias P2P entre usuarios o contactos con tasa aplicada y notas.

• **notificaciones:** Alertas y avisos en tiempo real para la campaña de notificaciones de cada usuario.

PREGUNTA 9**9. ¿Por qué se utilizó el tipo de dato UUID con `gen_random_uuid()` en lugar de enteros autoincrementales?**

Respuesta: Los UUID v4 ofrecen tres ventajas críticas: 1) Previenen ataques de enumeración donde un usuario malicioso adivina IDs secuenciales (ej. `/transacciones/101` vs `/transacciones/102`); 2) Permiten que clientes o subagentes generen identificadores únicos localmente sin colisiones antes de enviar al servidor; 3) Facilitan la replicación distribuida y la integración directa con `'auth.users'` de Supabase Auth, que opera nativamente con UUIDs.

PREGUNTA 10**10. ¿Qué tipos ENUM personalizados se definieron en PostgreSQL y qué valor aportan?**

Respuesta: Se crearon: `'rol_usuario'` (`'ADMIN'`, `'USUARIO'`), `'tipo_transaccion'` (`'ingreso'`, `'egreso'`), `'estado_transaccion'` (`'completada'`, `'pendiente'`, `'fallida'`) y `'estado_transferencia'` (`'pendiente'`, `'completada'`, `'rechazada'`). Aportan validación en tiempo de compilación dentro de PostgreSQL, previenen errores tipográficos en el código y optimizan el espacio en disco frente a cadenas de texto libres.

PREGUNTA 11**11. ¿Cómo se configuró la integridad referencial y qué efecto tiene `ON DELETE CASCADE` en las claves foráneas?**

Respuesta: Todas las tablas que dependen del usuario (`'transacciones'`, `'contactos'`, `'notificaciones'`, `'transferencias'`) declaran `'references public.usuarios(id) on delete cascade'`. Esto asegura que si un usuario es eliminado de la plataforma, todas sus entidades hijas asociadas se limpien automáticamente a nivel de base de datos, evitando registros huérfanos e inconsistencias relacionales.

PREGUNTA 12**12. ¿Por qué se utiliza el tipo `numeric(12, 2)` para los importes en lugar de `float` o `double`?**

Respuesta: Los tipos de coma flotante binaria (`'float'`, `'double'`) sufren de imprecisión en aritmética binaria (ej. $0.1 + 0.2 = 0.30000000000000004$), lo cual es inaceptable en sistemas bancarios y contables. El tipo `'numeric(12, 2)'` almacena números exactos en formato de base decimal fija, garantizando que sumas, egresos y conversiones de divisa no sufran pérdidas de centavos ni discrepancias contables.

PREGUNTA 13**13. ¿Qué restricciones defensivas `CHECK` se programaron en las tablas?**

Respuesta: Se aplicaron restricciones matemáticas e invariantes de negocio:

- En `'transacciones'` y `'transferencias'`: `'CHECK (monto > 0)'` o `'CHECK (monto_origen > 0)'`, impidiendo importes negativos o ceros.
- En `'tasas_cambio'`: `'CHECK (tasa > 0)'` impidiendo tasas inválidas.
- En códigos de moneda: `'char(3)'` asegurando apego estricto al estándar ISO 4217 (EUR, USD, GBP, PEN).

PREGUNTA 14**14. ¿Qué índices se crearon en PostgreSQL y por qué son compuestos?**

Respuesta: Se crearon índices clave como: 'idx_transacciones_usuario_fecha' sobre '(usuario_id, fecha DESC)', 'idx_contactos_usuario' sobre '(usuario_id, favorito DESC, nombre ASC)' y 'idx_notificaciones_usuario' sobre '(usuario_id, leida, creado_en DESC)'. Son compuestos porque la cláusula WHERE siempre filtra primero por el usuario activo y luego ordena por fecha o prioridad. Esto permite que PostgreSQL resuelva consultas complejas en tiempo $O(\log N)$ directamente desde el índice sin escanear la tabla completa ni ordenar en memoria RAM.

PREGUNTA 15**15. ¿Cuál es el propósito de la vista 'vista_transacciones_detalle'?**

Respuesta: Desnormaliza y une ('JOIN') las tablas 'transacciones', 'comercios' y 'usuarios'. Proporciona una consulta única y lista para el frontend con el nombre del comercio, logotipo/icono, categoría, nombre del usuario remitente y formato de fecha, evitando que la aplicación de Angular tenga que realizar múltiples peticiones en cascada para renderizar una fila de la tabla de historial.

PREGUNTA 16**16. ¿Cómo se modela la relación entre transacciones y comercios?**

Respuesta: La columna 'comercio_id' en 'transacciones' es una clave foránea hacia 'public.comercios(id)'. Cada comercio posee un nombre ('Starbucks', 'Netflix', 'Amazon', 'Payline'), una categoría ('Alimentación', 'Entretenimiento', 'Servicios') y una URL o clase de logotipo. Cuando la transacción es interna entre personas, se asocia al comercio 'Payline' con descripción personalizada del contacto.

PREGUNTA 17**17. ¿Cómo se gestionan las tasas de cambio multdivisa en la tabla 'tasas_cambio'?**

Respuesta: Cada fila almacena 'moneda_origen', 'moneda_destino', 'tasa' y 'actualizado_en'. Por ejemplo, para el par EUR/USD la tasa es 1.18, mientras que para el par inverso USD/EUR es 0.8475. Posee una restricción de unicidad '(moneda_origen, moneda_destino)', garantizando que no existan tasas duplicadas y permitiendo conversiones instantáneas con la función 'fn_obtener_tasa'.

PREGUNTA 18**18. ¿Qué función cumple el script de Node.js 'verificar-conexion.mjs'?**

Respuesta: Es una herramienta de prueba automatizada y de diagnóstico. Se conecta a Supabase mediante la clave anónima y verifica la disponibilidad de conexión, cuenta las filas de cada tabla, comprueba la lectura de la vista 'vista_transacciones_detalle' y ejecuta una llamada a la función RPC 'fn_resumen_usuario', validando que el entorno esté 100% operativo sin errores de sintaxis o permisos.

MÓDULO 3: SEGURIDAD, AUTENTICACIÓN Y ROW LEVEL SECURITY (RLS)

Políticas RLS, Supabase Auth, gestión de sesiones, guardas funcionales y permisos de base de datos.

PREGUNTA 19**19. ¿Qué es RLS (Row Level Security) y cómo protege los datos en este proyecto?**

Respuesta: RLS es una característica de seguridad nativa del motor de PostgreSQL que intercepta cada instrucción SQL (SELECT, INSERT, UPDATE, DELETE) y aplica filtros booleanos invisibles. En Payline, aunque un cliente malicioso intente solicitar 'SELECT * FROM transacciones', la política RLS asegura que PostgreSQL solo devuelva las filas donde el usuario tiene autorización, bloqueando por completo el acceso a los datos de otros clientes bancarios.

PREGUNTA 20**20. ¿Qué diferencia existe entre los roles de base de datos 'anon' y 'authenticated' en Supabase?**

Respuesta:

- **anon:** Representa peticiones realizadas por clientes que no han iniciado sesión o que usan la clave pública sin token JWT válido (utilizado en la pantalla de login/registro o para lectura de catálogos públicos como comercios).
- **authenticated:** Representa usuarios con un token JWT firmado por Supabase Auth tras iniciar sesión exitosamente. Permite aplicar políticas basadas en su identificador único autenticado ('auth.uid()').

PREGUNTA 21**21. ¿Cuál es la diferencia crítica entre un GRANT SQL y una política RLS (CREATE POLICY)?**

Respuesta: El 'GRANT' otorga el permiso técnico general para ejecutar la operación a nivel de tabla (ej. 'GRANT INSERT ON transacciones TO authenticated'). Sin embargo, tener el GRANT no es suficiente si RLS está activado: la política RLS define los predicados WHERE y WITH CHECK fila por fila. Si existe el GRANT pero no hay una política permisiva que evalúe a TRUE, PostgreSQL rechaza la operación inmediatamente con violación de RLS.

PREGUNTA 22**22. ¿Por qué ocurría inicialmente el error 'La política de seguridad (RLS) ha bloqueado la operación' al transferir?**

Respuesta: Ocurría por una doble causa: 1) La función PL/pgSQL 'fn_registrar_transferencia' no poseía la directiva 'SECURITY DEFINER', por lo que se ejecutaba con los privilegios limitados del invocador anónimo; 2) La tabla 'public.notificaciones' carecía del permiso de inserción 'GRANT INSERT' y de una política 'pol_notificaciones_insercion' para registrar el aviso en la campana. Al agregar 'SECURITY DEFINER' y la política de inserción correspondiente, la operación se completó atómicamente.

PREGUNTA 23**23. ¿Cómo interactúan 'auth.users' de Supabase Auth y la tabla de la aplicación 'public.usuarios'?**

Respuesta: 'auth.users' es la tabla interna y aislada de Supabase donde residen las credenciales criptográficas, contraseñas con hash bcrypt y metadatos de acceso. 'public.usuarios' es la tabla de perfiles de la aplicación con columnas del negocio (rol, color_avatar, saldo base). Cuando un usuario se registra o inicia sesión, la aplicación asegura la existencia del perfil en 'public.usuarios' utilizando como clave primaria el mismo UUID generado por 'auth.users'.

PREGUNTA 24**24. ¿Por qué es vital configurar 'persistSession: true' y 'autoRefreshToken: true' en el cliente de Supabase?**

Respuesta: Con 'persistSession: true', Supabase almacena el JWT y el Refresh Token en el localStorage del navegador del usuario, impidiendo que la sesión se destruya al recargar la página (F5). Con 'autoRefreshToken: true', el cliente renueva el token de acceso antes de que expire en segundo plano, evitando cierres de sesión abruptos o peticiones fallidas por expiración de credenciales.

PREGUNTA 25**25. ¿Cómo funcionan las guardas funcionales 'autenticacionGuard' y 'publicoGuard' en Angular 19?**

Respuesta: Son funciones basadas en la API 'CanActivateFn' inyectando 'AutenticacionServicio' y 'Router':

- **autenticacionGuard:** Verifica 'auth.estaAutenticado()'. Si hay sesión permite el paso; si no, redirige a '/login'.
- **publicoGuard:** Si el usuario ya está autenticado e ingresa manualmente a '/login', lo redirige directamente al panel principal ('/transacciones'), previniendo formularios redundantes.

PREGUNTA 26**26. ¿A qué se debe el error 'Email not confirmed' y cómo se soluciona en Supabase?**

Respuesta: Supabase Auth incluye por defecto la confirmación obligatoria de correo electrónico. Cuando un usuario se registra mediante 'signUp', se marca con 'email_confirmed_at = NULL' hasta que pulse el enlace enviado por correo. Para entornos de desarrollo o pruebas rápidas, se desactiva en el panel de Supabase: 'Authentication > Providers > Email > Confirm email (OFF)' o se ejecuta en SQL 'UPDATE auth.users SET email_confirmed_at = NOW()'.

PREGUNTA 27**27. ¿Qué es el 'Modo Demo' implementado en AutenticacionServicio y para qué sirve?**

Respuesta: Es un mecanismo de acceso rápido diseñado para evaluaciones, pruebas de interfaz y presentaciones de producto. Permite iniciar sesión con un solo clic como el usuario de demostración predeterminado (William Grace), cargando al instante todas sus transacciones sembradas, contactos y saldo sin necesidad de ingresar contraseñas.

PREGUNTA 28**28. ¿Cómo previene el SDK de Supabase ataques de inyección SQL?**

Respuesta: El cliente de Supabase no concatena texto para armar sentencias SQL; utiliza consultas parametrizadas internamente a través de PostgreSQL y tipos binarios en PostgreSQL. Todos los argumentos pasados en llamadas '.eq()', '.insert()' o '.rpc()' se transmiten como parámetros tipados independientes, haciendo imposible la inyección de código SQL malicioso.

MÓDULO 4: LÓGICA DE NEGOCIO Y FUNCIONES PL/PGSQL

Procedimientos almacenados en el servidor, transacciones atómicas, agregaciones y P2P.

PREGUNTA 29**29. ¿Qué es una función SECURITY DEFINER en PostgreSQL y por qué se usó en fn_registrar_transferencia?**

Respuesta: Una función normal en PostgreSQL se ejecuta con los privilegios del usuario que la invoca (SECURITY INVOKER). Una función 'SECURITY DEFINER' se ejecuta con los privilegios del superusuario o creador de la función. Se utilizó en 'fn_registrar_transferencia' porque una transferencia necesita insertar registros contables en 'transacciones' y en 'notificaciones' para dos usuarios distintos simultáneamente, superando las restricciones estándar del usuario emisor.

PREGUNTA 30**30. ¿Por qué es obligatorio incluir 'SET search_path = public' en funciones SECURITY DEFINER?**

Respuesta: Es una regla crítica de ciberseguridad en PostgreSQL recomendada por el equipo de seguridad de Supabase. Si no se especifica el 'search_path', un usuario malintencionado podría crear una tabla o función maliciosa con el mismo nombre en un esquema temporal con mayor prioridad, secuestrando la ejecución del superusuario ('search path hijacking'). Fijarlo en 'public' mitiga completamente esta vulnerabilidad.

PREGUNTA 31**31. ¿Cómo garantiza fn_registrar_transferencia la atomicidad (propiedad ACID)?**

Respuesta: En PostgreSQL, todo el cuerpo de una función en lenguaje PL/pgSQL ('BEGIN ... END;') se ejecuta dentro de una única transacción implícita. Si la conversión de moneda, la inserción del egreso o el aviso en notificaciones llega a fallar o lanzar una excepción ('RAISE EXCEPTION'), PostgreSQL ejecuta un ROLLBACK total automático. No existe posibilidad de que el dinero salga de una cuenta sin registrarse la transferencia.

PREGUNTA 32**32. ¿Cómo se diseñó la lógica de transferencia P2P bidireccional entre usuarios de Payline?**

Respuesta: La función 'fn_registrar_transferencia' examina el correo del contacto destinatario. Si el correo coincide con un usuario registrado en 'public.usuarios':

- 1) Registra la transacción de **egreso** en la cuenta del remitente.
 - 2) Registra automáticamente una transacción de **ingreso** ('+ Importe') con estado 'completada' en la cuenta del destinatario.
 - 3) Inserta una notificación en la campana del destinatario informando del dinero recibido con el nombre del remitente.
- Así, ambas cuentas ven sus balances e historiales sincronizados en tiempo real.

PREGUNTA 33**33. ¿Cómo calcula fn_resumen_usuario los totales de saldo, ingresos y egresos de forma optimizada?**

Respuesta: Utiliza cláusulas de agregación condicional de PostgreSQL ('FILTER WHERE'):

- total_ingresos: SUM(monto) FILTER (WHERE tipo = 'ingreso' AND estado = 'completada')
- total_egresos: SUM(monto) FILTER (WHERE tipo = 'egreso' AND estado = 'completada')
- saldo: total_ingresos - total_egresos

Esto resuelve todos los indicadores financieros del panel en un único escaneo de índice ('Single Scan') sin hacer 5 consultas separadas.

PREGUNTA 34**34. ¿Por qué PostgREST devuelve los campos numeric como cadenas de texto (string) en el JSON?**

Respuesta: JavaScript maneja los números como punto flotante de doble precisión según IEEE 754 (con un límite seguro de 53 bits en 'Number.MAX_SAFE_INTEGER'). Si PostgREST devolviera números decimales grandes o de alta precisión monetaria como números JSON, JavaScript podría redondearlos y alterar la cifra. Por seguridad estándar, PostgREST los serializa como cadenas ('1520.50'), y el frontend los convierte explícitamente con 'Number(valor)' o utilidades numéricas.

PREGUNTA 35**35. ¿Cómo calcula fn_volumen_pagos el volumen de transacciones de los últimos 7 días con días en español?**

Respuesta: Genera una serie continua de fechas con 'generate_series(CURRENT_DATE - INTERVAL '6 days', CURRENT_DATE, '1 day')' y realiza un LEFT JOIN con las transacciones de egreso completadas de ese usuario. Luego mapea el día de la semana a su abreviatura en español ('Lun', 'Mar', 'Mié', 'Jue', 'Vie', 'Sáb', 'Dom') y calcula el total gastado por día, garantizando que los días sin gastos figuren con 0 en el gráfico de barras.

MÓDULO 5: FRONTEND EN ANGULAR: COMPONENTES STANDALONE Y ARQUITECTURA

Componentes Standalone, control flow nativo, SVG en línea, lazy loading y modularidad.

PREGUNTA 36**36. ¿Qué beneficios aporta el enfoque de Componentes Standalone de Angular frente a los NgModules tradicionales?**

Respuesta: Los componentes Standalone eliminan la complejidad de los archivos 'module.ts' y las declaraciones cruzadas. Cada componente declara explícitamente sus dependencias en su metadato 'imports: [Tarjeta, Avatar, Icono, FormsModule]', favorece un 'tree-shaking' mucho más agresivo por parte del compilador Vite/esbuild y permite la carga perezosa directa en el enrutador sin módulos intermediarios.

PREGUNTA 37**37. ¿Cómo se configuró la carga perezosa (Lazy Loading) de páginas en app.routes.ts?**

Respuesta: Cada ruta principal ('/transacciones', '/login', '/informes', '/panel') se carga mediante 'loadComponent: () => import('./...').then(m => m.Componente)'. Esto fragmenta el código generado en pequeños chunks de JavaScript ('chunk-login.js', 'chunk-transacciones.js') que el navegador solo descarga cuando el usuario navega a esa sección, reduciendo drásticamente el peso inicial de la aplicación ('Initial Bundle').

PREGUNTA 38**38. ¿Qué es la sintaxis de Control Flow (@if, @for, @switch) y por qué supera a las directivas estructurales clásicas?**

Respuesta: Introducida nativamente en las últimas versiones de Angular, reemplaza a '*ngIf', '*ngFor' y '*ngSwitch'. Sus ventajas son: 1) No requiere importar 'CommonModule'; 2) Sintaxis ergonómica y limpia similar a JavaScript/TypeScript; 3) Soporta bloques integrados como '@empty' en bucles for para pintar estados vacíos sin condiciones adicionales; 4) Mayor velocidad de compilación y menor sobrecarga de memoria en el DOM.

PREGUNTA 39**39. ¿Cómo está implementado el componente Icono y por qué utiliza SVG en línea en lugar de librerías externas?**

Respuesta: El componente 'Icono' contiene un switch interno que dibuja paths de SVG vectoriales ('check', 'usuario', 'candado', 'correo', 'salir', etc.). Ventajas: 1) Cero peticiones de red adicionales; 2) No depende de fuentes tipográficas pesadas de iconos; 3) Adopta automáticamente el color del texto mediante 'currentColor'; 4) Es escalable a cualquier tamaño sin pixelarse y soporta renderizado accesible.

PREGUNTA 40**40. ¿Cómo funciona el componente Avatar para pintar iniciales y colores cuando no hay fotografía?**

Respuesta: Si 'url' es nula o indefinida, el componente ejecuta una función utilitaria que toma el nombre ('Adler Cisneros'), extrae las primeras letras de cada palabra ('AC') y aplica un color de fondo dinámico recibido como propiedad (ej. '#5B4DF0'). Si existe una URL válida, renderiza la etiqueta '' con manejo de errores de carga.

PREGUNTA 41**41. ¿Cuál es el propósito del componente Esqueleto (app-esqueleto) y qué aporta a la experiencia de usuario?**

Respuesta: Implementa un cargador tipo 'Skeleton Screen' que proyecta la silueta gris de las tarjetas, botones y filas de tablas mientras las peticiones asíncronas de Supabase están en curso. Esto reduce la percepción de tiempo de espera, previene saltos visuales en el layout ('Cumulative Layout Shift' o CLS) y ofrece una interfaz moderna y profesional.

PREGUNTA 42**42. ¿Cómo se estructuró el sistema de notificaciones flotantes con AvisosServicio y AvisosFlotantes?**

Respuesta: 'AvisosServicio' mantiene una señal reactiva 'readonly avisos = signal([]);' con métodos 'exito()', 'error()' e 'info()'. Cada aviso se autoelimina tras un temporizador de 4 segundos. El componente visual 'AvisosFlotantes' está anclado en la esquina superior de la pantalla mediante 'fixed' y escucha esta señal, renderizando alertas animadas con su icono correspondiente.

PREGUNTA 43**43. ¿Cómo está compuesta la página principal TransaccionesPagina y sus componentes hijos?**

Respuesta: Actúa como contenedor orquestador organizando tres componentes principales:

- 1) **TransferenciaRápida:** Fila de contactos, importe de envío, conversión en vivo, botón de arrastre y modal de destinatarios.
- 2) **HistorialTransacciones:** Tabla paginada con buscador en tiempo real, filtros de estado y formato de monedas.
- 3) **VolumenPagos:** Gráfico interactivo de barras de los últimos 7 días con tooltips y porcentajes calculados.

MÓDULO 6: GESTIÓN DEL ESTADO CON ANGULAR SIGNALS Y REACTIVIDAD

Signals, computed, effects, inmutabilidad y detección de cambios de grano fino.

PREGUNTA 44**44. ¿Qué son los Angular Signals (signal, computed, effect) y en qué superan a Zone.js tradicional?**

Respuesta: Los Signals son primitivas reactivas que gestionan valores con notificación sincrónica de dependencias. A diferencia de Zone.js, que intercepta todos los eventos asíncronos y verifica todo el árbol de componentes ('Dirty Checking'), los Signals informan exactamente qué nodo del DOM necesita actualizarse ('fine-grained reactivity'), eliminando trabajo innecesario de la CPU y mejorando el rendimiento.

PREGUNTA 45**45. ¿Por qué se utiliza el patrón private _dato = signal(...) y public dato = _dato.asReadOnly()?**

Respuesta: Es un principio fundamental de encapsulamiento e inmutabilidad en Clean Code. Los componentes de la interfaz pueden leer y suscribirse a 'dato' para renderizar la pantalla, pero no pueden alterar su valor arbitrariamente con '.set()'. Toda mutación del estado debe originarse a través de métodos controlados y validados del servicio ('cargar()', 'crear()', 'actualizar()').

PREGUNTA 46**46. ¿Cómo sincroniza UsuariosServicio los cambios de sesión usando effect() de Angular?**

Respuesta: En su constructor, 'UsuariosServicio' define un 'effect(async () => { ... })' que observa el id del usuario activo en 'AutenticacionServicio'. En el momento en que un usuario inicia sesión, cierra sesión o cambia a modo demo, el efecto se dispara automáticamente, invalidando los datos anteriores y disparando la recarga del perfil y del resumen financiero sin requerir recargar la página.

PREGUNTA 47**47. ¿Cómo se implementó la paginación y búsqueda reactiva en HistorialTransacciones?**

Respuesta: El componente define señales para 'paginaActual', 'tamanoPagina' (ej. 5 por página) y 'terminoBusqueda'. Mediante una señal calculada 'computed(() => { ... })', filtra primero las transacciones cuyo comercio o descripción coincida con el texto y luego aplica un '.slice(inicio, fin)'. Si la búsqueda reduce los resultados, la página se reajusta automáticamente al rango válido.

PREGUNTA 48**48. ¿Cómo calcula el widget de Transferencia Rápida la conversión en vivo usando computed()?**

Respuesta: Tiene como dependencias las señales 'montoOrigen()', 'monedaOrigen()', 'monedaDestino()' y las tasas de cambio del servicio. La señal calculada 'montoDestino = computed(() => round(montoOrigen() * tasa, 2))' recalcula instantáneamente el valor final que recibirá el contacto cada vez que el usuario modifica una sola tecla del input o cambia la moneda en el desplegable.

PREGUNTA 49**49. ¿Cómo se modeló la señal de estado de carga EstadoCarga en los servicios del núcleo?**

Respuesta: Se definió el tipo 'EstadoCarga = 'inactivo' | 'cargando' | 'listo' | 'error''. Todos los servicios exponen una señal 'estado = signal('inactivo')'. Esto permite a las vistas presentar estados deterministas y claros: mostrar esqueletos cuando es 'cargando', la tabla cuando es 'listo' o un mensaje de reintento amigable cuando es 'error'.

MÓDULO 7: UI/UX, INTERACCIÓN TÁCTIL, TOKENS SCSS Y ACCESIBILIDAD

Deslizante swipe-to-send, tokens CSS, modal accesible, responsive design y pointer events.

PREGUNTA 50**50. ¿Cómo funciona el mecanismo de arrastre (swipe-to-send) del deslizante de transferencias?**

Respuesta: Se basa en la API estándar de eventos Pointer ('pointerdown', 'pointermove', 'pointerup', 'pointercancel'). Al pulsar el tirador, se captura el puntero ('setPointerCapture') y se calcula la posición relativa X respecto al ancho de la pista deslizante. La posición se normaliza de 0 a 1 en la señal 'progreso()'. Al sobrepasar el 85% de la barra, se dispara la transferencia con confirmación deliberada.

PREGUNTA 51**51. ¿Qué error de doble ejecución ocurría al soltar el deslizante y cómo se solucionó?**

Respuesta: Al soltar el puntero tras un arrastre ('pointerup'), los navegadores web disparan sintéticamente un evento 'click' inmediatamente posterior en el mismo elemento. Si el usuario arrastraba y soltaba, el código ejecutaba el envío por arrastre y microsegundos después el 'click' volvía a intentar enviar. Se solucionó con una bandera 'haArrastrado' que absorbe y descarta cualquier evento 'click' residual si hubo movimiento.

PREGUNTA 52**52. ¿Cómo se diseñó el sistema de Tokens en SCSS mediante variables CSS nativas?**

Respuesta: En 'src/estilos/_variables.scss' se declaran tokens semánticos en el selector ':root' (ej. '--color-violeta: #5B4DF0', '--color-superficie: #FFFFFF', '--esp-4: 16px', '--radio-lg: 24px'). Las hojas de estilo de los componentes utilizan estas variables nativas. Esto permite que cambiar un color o implementar un tema oscuro (Dark Mode) sea instantáneo sin recompilar el código SCSS.

PREGUNTA 53**53. ¿Cómo está diseñado el modal para agregar nuevos destinatarios?**

Respuesta: Se implementó como un componente de diálogo flotante con fondo semitransparente ('backdrop-filter: blur(4px)'). Cuenta con dos pestañas: 'Usuarios en Payline' (consulta y muestra otros perfiles reales del sistema con botón 'Agregar' directo) e 'Ingresar manualmente' (formulario accesible con nombre, correo y moneda preferida). El modal valida los campos y enfoca el botón de cierre con accesibilidad de teclado (Escape).

PREGUNTA 54**54. ¿Qué prácticas de accesibilidad (a11y) se integraron en los componentes interactivos?**

Respuesta:

- Botones con etiquetas descriptivas 'aria-label' y títulos informativos.
- Los botones de selección de destinatarios alternan el atributo 'aria-pressed' al seleccionarse.
- El deslizante swipe-to-send es operable mediante teclado pulsando 'Espacio' o 'Enter'.
- Estilos de foco visible con contornos destacados ('@include foco-visible') para usuarios que navegan mediante tabulador.

MÓDULO 8: DEPURACIÓN, RESOLUCIÓN DE PROBLEMAS Y BUENAS PRÁCTICAS

Traducción de errores, configuración de budgets en angular.json y calidad de código.

PREGUNTA 55**55. ¿Cómo funciona el módulo centralizado de utilidades de errores (errores.util.ts)?**

Respuesta: Contiene un array de expresiones regulares que mapea mensajes técnicos de error de PostgreSQL o Supabase a lenguaje comprensible para usuarios:

- /row-level security/i → 'La política de seguridad (RLS) ha bloqueado la operación.'
- /Email not confirmed/i → 'Este correo no ha sido confirmado aún en Supabase.'
- /Invalid login credentials/i → 'Correo o contraseña incorrectos.'

Esto evita mostrar al usuario alertas crípticas en inglés o fugas de detalles internos del motor de base de datos.

PREGUNTA 56**56. ¿Por qué falló el build de Angular con un error de Budget de estilos y cómo se corrigió?**

Respuesta: Angular CLI impone por defecto límites estrictos de tamaño para los estilos de cada componente en 'angular.json' ('maximumWarning: 8kB', 'maximumError: 12kB'). Al incorporar las reglas visuales y animaciones del nuevo modal de destinatarios, 'transferencia-rapida.scss' alcanzó 12.15 kB (superando el límite por 146 bytes). Se corrigió ajustando los presupuestos de 'anyComponentStyle' a 16kB y 20kB en 'angular.json', logrando una compilación de producción completamente limpia.

PREGUNTA 57**57. ¿Qué buenas prácticas de TypeScript estricto se aplican en todo el proyecto?**

Respuesta: Se habilitó 'strict: true' en 'tsconfig.json'. Esto exige: 1) Prohibición de 'any' implícito; 2) Validación estricta de valores nulos o indefinidos ('strictNullChecks'); 3) Tipado exhaustivo de todas las respuestas de Supabase mediante interfaces dedicadas; 4) Inmutabilidad con 'readonly' en arreglos y propiedades que no deben mutar fuera de los servicios.

PREGUNTA 58**58. ¿Cuál es el procedimiento integral para desplegar o ejecutar este proyecto desde cero?**

Respuesta: 1) Ejecutar el archivo consolidado 'backend/supabase/instalacion_completa.sql' en el SQL Editor del proyecto en Supabase.
2) Desactivar la confirmación forzada de correo en 'Authentication > Providers > Email' si se desea acceso instantáneo.
3) Configurar las credenciales en 'frontend/src/environments/environment.ts'.
4) En la carpeta 'frontend', ejecutar 'npm install' y posteriormente 'npm start' o 'ng serve'.
5) Acceder a 'http://localhost:4200/login', ingresar con credenciales nuevas o utilizar el 'Modo Demo'.